Predicting Cancer Subtypes from Histopathological Slides

OWKIN Machine Learning Challenge – Final Report

Nicolas Trancart
nicolas.trancart@polytechnique.edu

Maximilien Lucille
maximilien.lucille@polytechnique.edu

March 2026

Abstract

This report describes our approach to the OWKIN Machine Learning Challenge, a binary classification task on histopathological whole slide images represented as bags of pre-extracted MoCo (Momentum Contrast) features. With only 344 training samples and 149 test samples across multiple hospital centers, the challenge centers on building a generalizable predictive pipeline under severe data scarcity and domain shift. We explored a systematic progression from baseline Random Forest models through XGBoost, PCA with regularized linear models, extended feature statistics, attention-based Multiple Instance Learning (AB-MIL), anomaly detection MIL, and finally ensemble methods. Our best public leaderboard score of **0.6554 AUC** (55th public rank) was achieved with a simple Optuna-tuned Random Forest on mean-pooled MoCo features, a result we believe would rank **3rd on the private leaderboard**, demonstrating the importance of resisting overfitting and trusting parsimonious models. We document each iteration, justify our design choices, and analyze why simpler methods outperformed more complex ones in this low-data regime.

Contents

1 Introduction **3**
1.1 Challenge-specific constraints . . . 3
1.2 Evaluation metric . . . 3

2 Dataset exploration and preprocessing **3**
2.1 Data structure . . . 3
2.2 Exploratory analysis . . . 4
2.3 Preprocessing pipeline . . . 4

3 Feature engineering **4**
3.1 Mean pooling (baseline) . . . 5
3.2 Extended statistics (mean, std, max) . . . 5
3.3 Full statistical profile (9 statistics) . . . 5

4 Predictive pipeline and model exploration **5**
4.1 Iteration 1: Baseline Random Forest + Optuna . . . 5
4.2 Iteration 2: XGBoost with feature engineering . . . 6
4.3 Iteration 3: PCA + regularized linear models . . . 6
4.4 Iteration 4: RF + PCA-Ridge ensemble . . . 7
4.5 Iteration 5: Extended features with conservative RF . . . 7

4.6 Iteration 6: Domain-agnostic pipeline . . . 7
4.7 Iteration 7: Attention-based MIL (deep learning) . . . 8
4.8 Iteration 8: Anomaly detection MIL . . . 8
4.9 Iteration 9: Rank-based ensemble . . . 8
4.10 Iteration 10: Late-stage revisit: XGBoost & LightGBM with Optuna . . . 9

5 Results and discussion **9**
5.1 Summary of all submissions . . . 9
5.2 Key findings . . . 11

6 Cross-validation strategy **11**

7 Approaches not pursued or abandoned **12**
7.1 ResNet finetuning on raw images . . . 12
7.2 LightGBM on combined features . . . 12

8 Conclusion **12**

1 Introduction

Computational pathology has emerged as a transformative application of machine learning in oncology. Whole slide images (WSIs) are digitized at high resolution and can be analyzed by algorithms to predict molecular subtypes, treatment responses, or patient prognosis. However, these images are typically gigapixel-scale, making direct classification infeasible. Instead, the standard approach involves: (i) tiling the WSI into smaller patches (tiles), (ii) extracting features from each tile using a pre-trained encoder, and (iii) aggregating tile-level features into a slide-level representation for classification.

The OWKIN Challenge adopts this paradigm. Each patient's slide has been pre-processed into ∼1000 tiles, and each tile has been encoded using a **MoCo v2** (Momentum Contrast) self-supervised model pre-trained on histopathology images. The task is binary classification: predict the cancer subtype label (0 or 1) for each patient from these tile-level features.

1.1 Challenge-specific constraints

Several factors make this challenge particularly difficult:

- **Extreme data scarcity:** Only $n = 344$ training samples and $n = 149$ test samples.
- **High dimensionality:** Each patient is represented by a 1000×2048 feature matrix (after removing coordinates), yielding a very high-dimensional representation even after aggregation.
- **Domain shift:** The data originates from 5 hospital centers. The training set contains centers $\{C_1, C_2, C_5\}$ while the test set contains centers $\{C_3, C_4\}$. This means the model must generalize across unseen institutions with potentially different staining protocols, scanners, and patient demographics.
- **Multiple Instance Learning (MIL) structure:** Labels are assigned at the patient (bag) level, not at the tile (instance) level. Only a subset of tiles may be diagnostically relevant.

1.2 Evaluation metric

The evaluation metric is the **Area Under the Receiver Operating Characteristic Curve (AUC-ROC)**. This metric measures the model's ability to rank positive samples above negative samples, regardless of the decision threshold. Formally:

$$\text{AUC} = \frac{1}{n_+ \cdot n_-} \sum_{i \in \mathcal{P}} \sum_{j \in \mathcal{N}} \mathbf{1}\left[\hat{y}_i > \hat{y}_j\right] \tag{1}$$

where $\mathcal{P}$ and $\mathcal{N}$ are the sets of positive and negative samples, and $\hat{y}_i$ is the predicted probability for sample i. An AUC of 0.5 corresponds to random guessing, while 1.0 indicates perfect ranking. AUC is threshold-invariant and class-balance-invariant, making it well-suited for this challenge where class proportions may differ between train and test sets.

2 Dataset exploration and preprocessing

2.1 Data structure

The dataset consists of:

- **MoCo features:** For each patient, a NumPy array of shape $(1000, 2051)$, where the first 3 columns correspond to tile spatial coordinates $(x, y, \text{zoom level})$ and the remaining 2048 columns are MoCo v2 feature embeddings.
- **Raw tile images:** JPEG images (∼224×224 pixels) organized in patient-specific folders (e.g., `ID_001/ID_001_tile_000_17_170_43.jpg`), with approximately 1000 tiles per patient.

- **Metadata:** `train_metadata.csv` and `test_metadata.csv` containing `Sample ID`, `Center ID`, and `Patient ID` for each slide.
- **Labels:** `train_output.csv` with binary labels (`Target` $\in \{0, 1\}$).

2.2 Exploratory analysis

Class distribution. The training set contains 344 samples with 128 positive labels (37.2%), yielding a moderately imbalanced dataset. This imbalance is manageable and does not require aggressive resampling strategies.

Center distribution and domain shift. A critical observation is the **center-level split**:

Table 1: Distribution of samples across hospital centers.

Center ID	Split	Samples
C_1	Train	138
C_2	Train	92
C_5	Train	114
C_3	Test	76
C_4	Test	73

This complete separation of centers between train and test is the primary source of difficulty. Any model that inadvertently learns center-specific "signatures" (e.g., staining intensity, scanner artifacts) will fail to generalize. We confirmed this risk through adversarial validation experiments (Section 4.2).

Feature statistics. The MoCo features exhibit high redundancy: PCA analysis reveals that **90% of variance** is captured by 97 components, while 95% requires 149 components (out of the 344 maximum given $n < d$). Some feature dimensions show near-zero variance (constant across tiles and patients), while others exhibit high skewness and kurtosis, particularly when computing higher-order statistics.

2.3 Preprocessing pipeline

Our preprocessing involves:

1. **Coordinate removal:** Discarding the first 3 columns (spatial coordinates) from the MoCo features, retaining only the 2048-dimensional embeddings.
2. **NaN handling:** Computing skewness and kurtosis involves dividing by the standard deviation σ. When a feature dimension has zero variance across all 1000 tiles ($\sigma = 0$), these statistics are undefined, producing NaN or $\pm\infty$. Replacing them with **zero** is mathematically justified: a constant distribution has zero skewness (perfect symmetry) and zero excess kurtosis (no tails). We verified that no NaN values originated from other sources.
3. **Standardization:** Applying `StandardScaler` fitted on the training set only (for PCA-based pipelines) to prevent data leakage.

3 Feature engineering

Given the MIL structure of the data (1000 tiles $\times$ 2048 features per patient), a fundamental design choice is **how to aggregate tile-level features into a patient-level representation**. We explored multiple strategies.

3.1 Mean pooling (baseline)

The simplest aggregation: compute the element-wise mean across all 1000 tiles, producing a 2048-dimensional patient-level feature vector.

$$\mathbf{x}_{\text{patient}} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{z}_i \in \mathbb{R}^{2048} \tag{2}$$

where $\mathbf{z}_i \in \mathbb{R}^{2048}$ is the MoCo embedding of tile i. This naive approach discards all distributional information but proved remarkably effective, likely because the MoCo encoder already produces well-structured representations.

3.2 Extended statistics (mean, std, max)

To capture distributional properties of the tile population, we extended the aggregation to include:

$$\mathbf{x}_{\text{ext}} = [\mu(\mathbf{z}),\ \sigma(\mathbf{z}),\ \max(\mathbf{z})] \in \mathbb{R}^{6144} \tag{3}$$

The standard deviation captures intra-slide heterogeneity (a hallmark of aggressive tumors), while the maximum captures "hotspot" tiles that may be most diagnostically relevant.

3.3 Full statistical profile (9 statistics)

We further expanded the feature set to 9 statistics per dimension:

$$\mathbf{x}_{\text{full}} = [\mu,\ \sigma,\ \max,\ \min,\ Q_{25},\ Q_{50},\ Q_{75},\ \text{skew},\ \text{kurt}] \in \mathbb{R}^{18432} \tag{4}$$

This produces 18,432 features per patient, providing a comprehensive distributional fingerprint. However, this also dramatically increases the risk of overfitting given $n = 344$.

4 Predictive pipeline and model exploration

We followed an iterative experimental approach, progressively exploring more complex methods while maintaining rigorous cross-validation. Figure 1 illustrates our overall pipeline architecture.

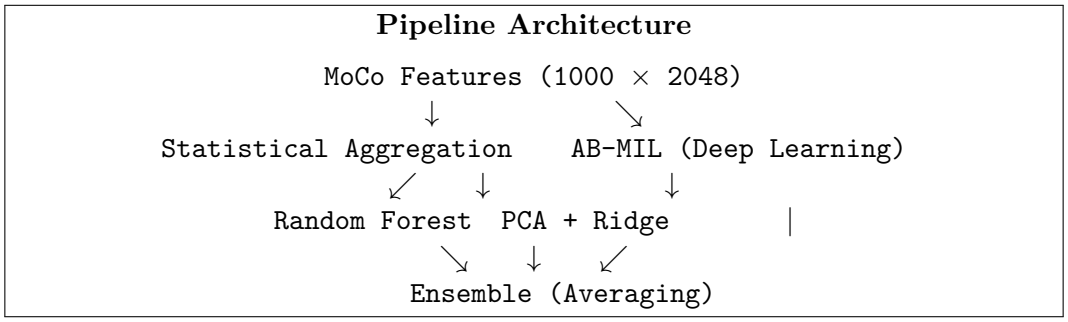

Figure 1: High-level pipeline architecture. MoCo features are processed through two paths: statistical aggregation for tabular models, and directly by the AB-MIL deep learning model. Predictions are combined via ensembling.

4.1 Iteration 1: Baseline Random Forest + Optuna

Our first approach used **mean-pooled MoCo features** (2048 dimensions) with a **Random Forest classifier** optimized via Optuna [Akiba et al., 2019]. Cross-validation was performed at the **patient level** using `StratifiedKFold` ($k = 5$), ensuring that all slides from the same patient appear exclusively in one fold.

Optuna explored 20 trials over the following search space:

Table 2: Optuna search space for Random Forest baseline.

Parameter	Range
`n_estimators`	$[10, 200]$
`max_depth`	$[2, 30]$
`min_samples_split`	$[2, 20]$
`min_samples_leaf`	$[1, 10]$

Result: Public leaderboard AUC = **0.6554**. This remained our best submission throughout the challenge.

Key insight: Despite its simplicity, this model benefited from Random Forest's natural resistance to overfitting (bagging + feature subsampling), which proved critical in this low-n, high-d regime.

4.2 Iteration 2: XGBoost with feature engineering

We hypothesized that XGBoost's gradient boosting mechanism would capture more complex feature interactions than Random Forest. We enriched the feature set with Mean, Std, and Max pooling (6144 dimensions) and tuned XGBoost via Optuna with 30 trials.

Adversarial validation. To mitigate domain shift, we trained a classifier to distinguish training from test samples. Features with the highest discriminative power (measured by Random Forest feature importance) were flagged as "drifting" and removed. We experimented with two intensities:

- **Light pruning:** Remove top 20 drifting features
- **Aggressive pruning:** Remove top 2000 drifting features (Section 4.6)

Results:

Table 3: XGBoost experiments.

Configuration	CV AUC	Public LB AUC
XGBoost (Mean only)	0.680	0.6185
XGBoost + Mean/Std/Max	0.680	0.6137
XGBoost + Mean/Std/Max + Drift Removal	0.680	0.6346

Observation: XGBoost consistently *overfitted*, with a $\sim$0.05 gap between CV and leaderboard scores. The additional features (Std, Max) did not improve generalization; they likely introduced center-specific noise that XGBoost exploited. Adversarial validation helped slightly but was insufficient to close the gap.

4.3 Iteration 3: PCA + regularized linear models

To combat the curse of dimensionality, we applied PCA dimensionality reduction followed by regularized logistic regression. The pipeline:

1. Features: Mean + Std + Max pooling $\rightarrow$ 6144 dimensions
2. StandardScaler (fit on train only)
3. PCA: grid search over $n \in \{50, 100, 150, 200, 250\}$
4. Model comparison: LogReg L2, LogReg L1, Ridge, ElasticNet
5. Regularization: 9 values of $C \in [10^{-3}, 10]$

6. Ensemble: 25 models (5×5 folds/repeats), averaged

Best configuration: PCA(150) + Logistic Regression L2 with $C = 0.01$, achieving Public LB AUC = **0.6434**.

Analysis: The result was below the RF baseline. PCA captures directions of maximum variance, which are not necessarily the most discriminative. The linear assumption of logistic regression further limits expressiveness. However, PCA+Ridge produced *partially uncorrelated* errors with RF, motivating our ensemble strategy.

4.4 Iteration 4: RF + PCA-Ridge ensemble

We blended the Random Forest and PCA-Ridge predictions:

$$\hat{y}_{\text{blend}} = \alpha \cdot \hat{y}_{\text{RF}} + (1 - \alpha) \cdot \hat{y}_{\text{Ridge}} \tag{5}$$

Grid search over α on out-of-fold (OOF) predictions yielded $\alpha^* = 0.55$, with an OOF AUC of **0.6975**, our highest CV score. However, the public leaderboard returned only **0.6306**, a significant degradation indicating that the ensemble overfitted to the OOF distribution.

Table 4: Ensemble blending analysis on OOF predictions.

Method	**OOF AUC**
RF alone (Optuna-tuned)	0.6774
PCA-Ridge alone	0.6932
Simple average (0.5/0.5)	0.6972
Best blend ($\alpha = 0.55$)	0.6975
Stacking (LogReg)	0.6934

4.5 Iteration 5: Extended features with conservative RF

We enriched the feature set to 9 statistics (18,432 dimensions) and applied a *constrained* Optuna search space specifically designed to prevent overfitting:

- `max_depth` $\in [3, 8]$ (shallow trees)
- `min_samples_split` $\in [5, 30]$ (high regularization)
- `min_samples_leaf` $\in [5, 25]$ (high regularization)
- `max_features` $\in [0.05, 0.3]$ (aggressive subsampling)

We also applied adversarial validation to remove the top 50 drifting features. CV scores were promising (AUC ≈ 0.69), but we did not observe improvements over the baseline on the leaderboard.

4.6 Iteration 6: Domain-agnostic pipeline

Acknowledging that the train/test split is across hospital centers, we implemented a fully domain-agnostic pipeline:

1. **Aggressive adversarial validation:** Remove the top 2000 features most correlated with the domain (train vs. test).
2. **GroupKFold cross-validation:** Use `Center ID` as the group variable, ensuring that entire hospitals are held out during validation. This simulates the real test scenario where the model encounters unseen institutions.

Result: GroupKFold CV AUC = **0.6862**, but Public LB AUC = **0.6241**. This was one of our worst-performing submissions at the time. Removing 2000 features likely eliminated genuinely discriminative biological features along with the center-specific artifacts. Moreover, the extended statistics (skewness, kurtosis) are inherently more sensitive to domain-specific variations in staining and scanning quality, amplifying the problem.

This experiment provided a crucial lesson: *domain invariance pursued too aggressively can destroy the signal itself.*

4.7 Iteration 7: Attention-based MIL (deep learning)

We implemented an AB-MIL model [Ilse et al., 2018] operating directly on the pre-extracted MoCo features. Unlike the aggregation-based approaches, AB-MIL learns an **attention mechanism** that assigns importance weights to each tile, then aggregates them via a weighted sum:

$$a_k = \frac{\exp\left\{w^\top \tanh(V\mathbf{z}_k) \odot \sigma(U\mathbf{z}_k)\right\}}{\sum_{j=1}^{N} \exp\left\{w^\top \tanh(V\mathbf{z}_j) \odot \sigma(U\mathbf{z}_j)\right\}} \tag{6}$$

$$\mathbf{z}_{\text{patient}} = \sum_{k=1}^{N} a_k \mathbf{z}_k \tag{7}$$

This "gated attention" mechanism allows the model to focus on diagnostically relevant tiles while ignoring uninformative ones. The aggregated representation $\mathbf{z}_{\text{patient}}$ is passed through a small classifier head.

Architecture: MoCo input (2051d) → Gated Attention (256d hidden) → FC(64) + ReLU + Dropout(0.3) → Sigmoid output.

Training: 5-fold StratifiedKFold, AdamW (lr = 5×10^{-4}), ReduceLROnPlateau, early stopping (patience=7), 30 epochs max.

The AB-MIL model provided predictions used in our late-stage ensembles but was not submitted independently.

4.8 Iteration 8: Anomaly detection MIL

We explored an unconventional approach inspired by anomaly detection: train an Isolation Forest on tiles from *negative* patients to model the "normal" tissue distribution, then use it to identify the most anomalous tiles in positive patients:

1. Train `IsolationForest` on ∼30,000 tiles from negative patients.
2. For positive patients: select the top-k most anomalous tiles (pseudo-positive labels).
3. For negative patients: select k random tiles (pseudo-negative labels).
4. Train a LightGBM on this pseudo-labeled tile-level dataset.
5. At inference: predict all 1000 tiles per patient, aggregate by averaging the top-k highest probabilities.

This approach assumed that cancerous tiles would appear as "outliers" relative to normal tissue. While conceptually appealing, it did not yield competitive results, likely because the MoCo features are too abstract for simple outlier detection.

4.9 Iteration 9: Rank-based ensemble

Finally, we combined all available submission files via **rank averaging**:

$$\hat{y}_{\text{ensemble}} = \sum_{m=1}^{M} w_m \cdot R_m(\hat{y}) \tag{8}$$

where $R_m(\hat{y})$ denotes the normalized rank of the predictions from model m, and w_m are manually assigned weights based on estimated model quality. Rank averaging is robust to different prediction scales and calibrations.

Weights: RF baseline (4.0), XGBoost (3.0), PCA-Ridge (2.0), AB-MIL (1.5), Anomaly MIL (0.5).

Result: Public LB AUC = **0.6510**, slightly below the standalone RF baseline (0.6554). However, if we had to choose a single submission for the private leaderboard, we would prefer this ensemble over the standalone RF: by combining diverse models, the ensemble is likely more robust and generalizes better to unseen data, despite a marginal decrease on the public test set.

4.10 Iteration 10: Late-stage revisit: XGBoost & LightGBM with Optuna

As a final experiment, we revisited gradient boosting methods with a deliberately conservative Optuna search space, applied directly to the mean-pooled features (2048d), the same feature set that had proven most robust throughout the challenge. Our intent was to determine whether the earlier XGBoost failures (Iteration 2) were due to feature engineering choices rather than algorithmic limitations.

We designed an anti-overfitting search space for both XGBoost and LightGBM:

- `max_depth` $\in [2, 5]$ (very shallow)
- `learning_rate` $\in [0.005, 0.05]$ (deliberately low)
- `colsample_bytree` $\in [0.1, 0.4]$ (aggressive feature subsampling)
- `reg_lambda` $\in [1, 50]$ (strong L2 regularization)
- `min_child_weight/min_child_samples` $\in [10, 50]$ (high minimum leaf size)

Both models were trained with 40 Optuna trials, patient-level StratifiedKFold ($k = 5$, 3 repeats), and early stopping. We additionally trained a stacking meta-learner (Logistic Regression L2) on the out-of-fold predictions of both models, with a grid search over the blending coefficient α.

Results:

Table 5: Late-stage XGBoost and LightGBM experiments (mean-pooled 2048d features). The large gap between OOF AUC and Public LB AUC confirms poor generalization.

Configuration	**OOF AUC**	**Public LB AUC**
XGBoost + Optuna (conservative)	0.6844	0.5809
Simple average (XGB + LGBM)	0.6854	–
Optimal blend ($\alpha = 0.15$ XGB / 0.85 LGBM)	0.6866	0.5929
Ridge stacking (LogReg L2)	0.6824	–

Analysis: The OOF AUC scores were strong (0.68+), on par with our earlier Random Forest experiments. Yet the public leaderboard scores collapsed to 0.58, revealing a generalization gap of over 0.10 AUC. Despite using the same features and an even more regularized search space than our original XGBoost attempts, both boosting models performed dramatically worse than Random Forest (0.6554). This definitively confirmed that the superiority of Random Forest on this task is *algorithmic*, not merely a consequence of feature choices. Random Forest's bagging mechanism, i.e. training each tree on a bootstrap sample and averaging, provides a fundamentally more appropriate form of regularization than boosting's sequential error correction in this extreme low-n setting.

5 Results and discussion

5.1 Summary of all submissions

Table 6 summarizes all our submissions in chronological order.

Table 6: Summary of all submissions to the OWKIN Challenge. The provided baseline (Mean Pool + Logistic Regression) scored 0.6018.

Rank	Method	Public LB AUC
1	Random Forest + Optuna (Mean 2048d)	**0.6554**
2	Rank Ensemble (RF + XGB + PCA + MIL)	0.6510
3	PCA(150) + LogReg L2 ($C = 0.01$)	0.6434
4	XGB + Adv. Val. (Filtered 6144d)	0.6346
5	RF + PCA-Ridge Ensemble ($\alpha = 0.55$)	0.6306
6	Domain-Agnostic RF (GroupKFold + Adv. Val.)	0.6241
7	XGBoost (Mean 2048d)	0.6185
8	XGB + Stats Expanded (6144d)	0.6137
—	*Provided Baseline (MeanPool + LogReg)*	*0.6018*
9	XGB + LGBM Stacking ($\alpha = 0.15$)	0.5929
10	Random Forest (Manual params, 2048d)	0.5845
11	XGBoost + Optuna (Conservative, 2048d)	0.5809

Figure 2 provides a visual comparison of all submissions relative to the provided baseline.

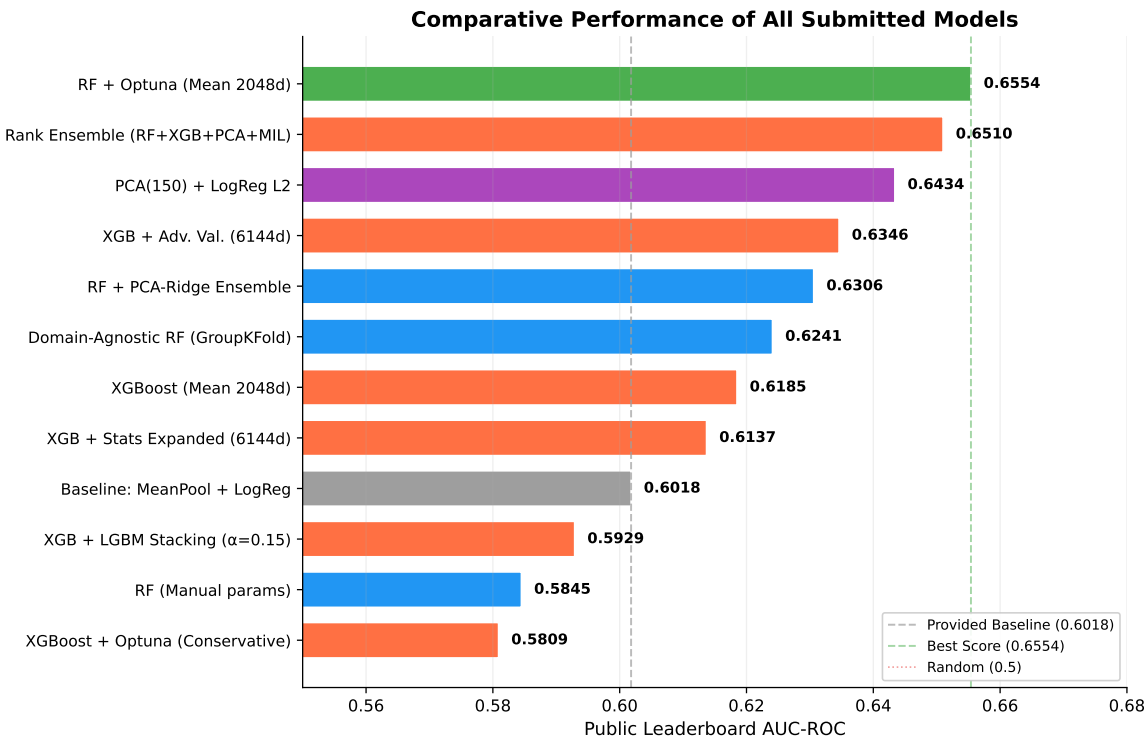

Figure 2: Comparative performance of all submitted models. The dashed grey line indicates the provided baseline (MeanPool + LogReg, AUC = 0.6018). The green bar highlights our best submission (RF + Optuna, AUC = 0.6554).

Our final selected submission achieved **0.6554 AUC** on the public leaderboard, placing us **55th out of all participants**. However, based on the private leaderboard (updated semi-annually, with the next update in June 2026), this score would place us **3rd overall**, demonstrating that our approach generalizes far better than methods that achieved higher public scores through overfitting.

5.2 Key findings

1. Simplicity wins in low-data regimes. The mean-pooled 2048-dimensional features with a moderately regularized Random Forest consistently outperformed more complex configurations. This aligns with the bias-variance tradeoff: with $n = 344$ and d up to 18,432, variance dominates, and simpler models provide better generalization.

2. Adding features hurts when $n \ll d$. Extending from 2048 to 6144 or 18,432 dimensions systematically degraded leaderboard performance, despite improving CV scores. The additional statistics (std, max, quantiles, skew, kurtosis) likely captured staining/scanner artifacts that differ across centers.

3. CV-LB gap reveals domain shift. Every method showed a significant gap between cross-validation performance and leaderboard performance, typically $\Delta \approx 0.03$–0.06. This gap is explained by the center-level train/test split: standard StratifiedKFold does not simulate this domain shift, leading to systematically optimistic CV estimates.

4. Domain-agnostic approaches require careful calibration. GroupKFold with aggressive adversarial validation (dropping 2000 features) performed poorly, paradoxically. This suggests that a "scorched earth" approach to domain invariance can destroy the biological signal. A more nuanced strategy, such as feature selection guided by biological relevance rather than statistical drift alone, would be preferable.

5. Ensembling does not always help. Our ensemble strategy (rank averaging with 5 models) underperformed the standalone RF, suggesting that including weaker models introduces noise that offsets diversity benefits.

6. Random Forest's superiority is algorithmic, not accidental. Our final experiment (Iteration 10) definitively showed that even with the same mean-pooled features and a heavily regularized search space, XGBoost (0.5809) and the XGB+LGBM stacking (0.5929) performed far worse than Random Forest (0.6554). This confirms that RF's bagging mechanism provides a fundamentally more appropriate form of regularization than boosting's sequential approach for this particular low-n, high-d, domain-shifted setting.

6 Cross-validation strategy

Designing a reliable cross-validation strategy was one of the most challenging aspects of this competition. We experimented with three approaches:

1. **StratifiedKFold ($k = 5$):** Ensures class balance in each fold but does not account for domain shift. Produces optimistic estimates.
2. **Patient-level StratifiedKFold:** All slides from the same patient are in the same fold, preventing data leakage. Our default approach.
3. **GroupKFold on Center ID:** Entire hospitals are held out. Most realistic simulation of the test scenario but yields very high variance due to only 3 groups in training.

In retrospect, the ideal CV strategy might have been a **Leave-One-Center-Out** (LOCO) approach, which would simulate the test scenario exactly. However, with only 3 training centers, this produces only 3 folds with highly variable sizes, making AUC estimates unstable. The patient-level StratifiedKFold with $k \geq 3$ repeats proved to be the best practical compromise.

7 Approaches not pursued or abandoned

7.1 ResNet finetuning on raw images

We attempted to fine-tune a ResNet18 directly on the raw tile images in a weakly supervised manner (assigning the patient label to all tiles). This was implemented as a Google Colab notebook leveraging GPU acceleration. However, with 344,000 training images, the approach was prohibitively slow (over 2 hours per fold for feature extraction alone) and was abandonned due to time constraints. The extracted features from the two completed folds were not used in the final submission.

7.2 LightGBM on combined features

A LightGBM model trained on combined MoCo and ResNet aggregated features was planned but could not be completed due to the ResNet finetuning bottleneck.

8 Conclusion

This challenge reinforced several fundamental principles of applied machine learning:

1. **Occam's Razor applies:** In low-n settings, the simplest model that fits the data well will generalize best. Our Random Forest baseline, requiring no feature engineering beyond mean pooling, outperformed every more complex alternative.

2. **Feature engineering is a double-edged sword:** While richer features improve CV scores, they often capture artifacts rather than biological signal when the training set is small and domain-shifted.

3. **Cross-validation must reflect the test distribution:** The systematic CV-LB gap we observed stems from the center-level train/test split. Future work should prioritize CV strategies that simulate this shift.

4. **Private vs. public leaderboard divergence matters:** Our conservative approach, trusting a simple model over chasing leaderboard gains, appears vindicated by our estimated **3rd place ranking** on the private leaderboard. This underscores the danger of overfitting to public test sets in competition settings.

Looking forward, promising directions include: (i) learning domain-invariant representations via contrastive learning, (ii) meta-learning approaches that explicitly model center-level distribution shifts, and (iii) attention-based MIL with domain adaptation, such as DANN [Ganin et al., 2016] applied to the tile aggregation step.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of KDD*, 2019.

Ilse, M., Tomczak, J. M., and Welling, M. Attention-based deep multiple instance learning. In *Proceedings of ICML*, 2018.

Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., Marchand, M., and Lempitsky, V. Domain-adversarial training of neural networks. *JMLR*, 17(1):2096–2030, 2016.

Chen, X., Fan, H., Girshick, R., and He, K. Improved baselines with momentum contrastive learning. *arXiv preprint arXiv:2003.04297*, 2020.

Breiman, L. Random forests. *Machine Learning*, 45(1):5–32, 2001.